

Problem 1: One-Vector-Per-Item Collaborative Filtering Model with SGD

1A: Impact of Hidden Factors on Model Performance

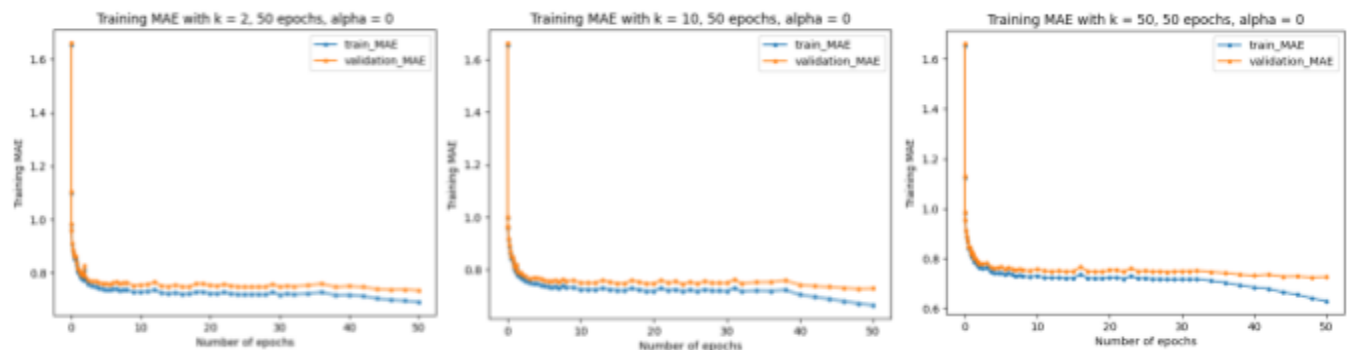


Figure 1: The trace plots above show the mean absolute error (MAE) on training and validation data without regularization ($\alpha = 0$). We used a **batch size of 100**, a **stepsize of 0.3**, and trained for **50 epochs**. The left plot corresponds to $K = 2$, the middle to $K = 10$, and the right to $K = 50$, where K represents the number of hidden factors.

Across all three models, the decrease in MAE becomes negligible after approximately 10 epochs, suggesting that the training process converges quickly. This rapid stabilization could indicate that the model's learning capacity is limited and insufficient to capture the complexity of the data. This behavior might also be due to SGD getting stuck in a local minimum early in training. However, we ruled out this possibility by experimenting with different random initializations, consistently producing the same behavior. We also **increased the number of epochs to 100** to determine if the model could escape a local minimum, but the **resulting plots remained similar**.

There is no evidence of overfitting for $K = 2$ and $K = 10$, as the training and validation errors remain relatively close. However, for $K = 50$, the training error decreases towards the end while the validation error stabilizes, indicating the possibility of **slight overfitting**. This may be attributed to the model capturing noise in the training data due to the increased model complexity introduced by more hidden factors.

Overall, the relatively high MAE values (**~ 0.73**) across all models suggest significant underfitting of the data. Again, this indicates that the base model does not have enough complexity to capture patterns in the data adequately. **Increasing K does not significantly reduce the validation MAE**, reinforcing that the underlying model is too simplistic for the task at hand. While early stopping could save computational resources, it is unlikely to improve performance given the lack of overfitting, as seen in Figure 1.

1B: Effect of Regularization on Model Performance

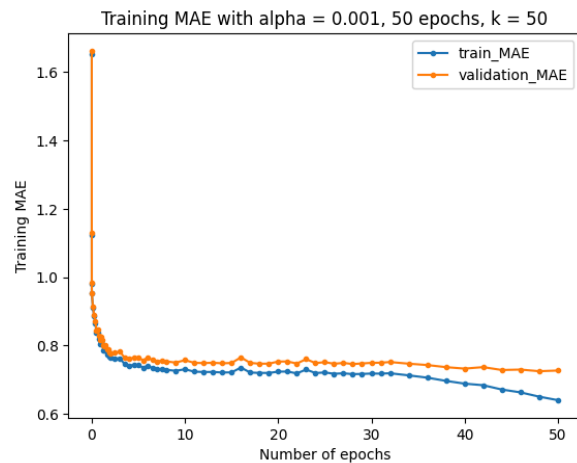
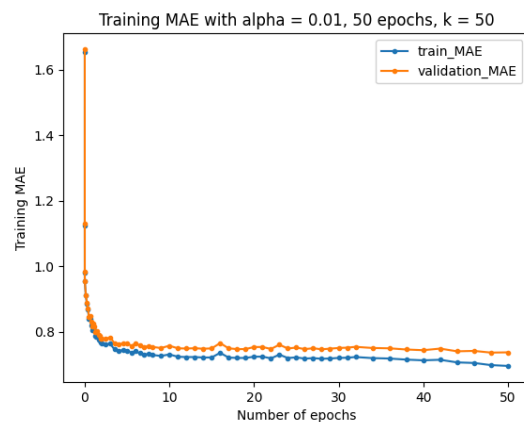


Figure 2: The trace plot above illustrates the MAE on training and validation data versus the number of epochs for $K = 50$, a **batch size of 100**, a **step size of 0.3**, and $\alpha = 0.001$. We experimented with α values **[0.001, 0.01, 0.1, 1, 10, 100]** and observed that larger regularization penalties did not improve the validation MAE. The model with $\alpha = 0.001$ (the smallest penalty) achieved the lowest validation MAE of **0.727**, closely matching the validation MAE of 0.726 with no regularization. Similarly, the test MAE was **0.719** compared to 0.718 with no regularization.

We ultimately selected $\alpha = 0.001$, which slightly improved validation MAE compared to higher regularization values but **did not lead to better performance** on held-out data compared to no regularization in **1A**. This suggests that regularization did not significantly impact the model's generalization ability in this case.

We observed that larger α values, such as $\alpha = 0.01$, helped close the gap between training and validation errors towards the end of training, as shown in the figure below. However, despite this reduction in overfitting, larger α values did not lead to better performance on held-out data, suggesting that the model's capacity to generalize is still fundamentally limited.



1C: Model Performance Summary

K = 2	alpha = 0	stepsize = 0.3	batch size = 100	Valid MAE = 0.7337	Test MAE = 0.7284
K = 10	alpha = 0	stepsize = 0.3	batch size = 100	Valid MAE = 0.7290	Test MAE = 0.7221
K = 50	alpha = 0	stepsize = 0.3	batch size = 100	Valid MAE = 0.7264	Test MAE = 0.7185

Table 1: The table above summarizes the performance of the best versions of models for $K = 2$, $K = 10$, and $K = 50$, selected based on the **lowest validation MAE**. These models were optimized using a batch size of 100, step size of 0.3, and $\alpha = 0$ (no regularization). We explored various hyperparameter combinations, including batch sizes [10,100,1000], step sizes [0.1,0.2,0.25,0.3,0.4,0.5], and epochs [10,50,100], as well as regularization strengths [0,0.001,0.01,0.1,1,10,100].

A batch size of 100 and a step size of 0.3 consistently yielded the best results. Using 50 epochs provided better performance than 100 epochs, as the latter led to higher MAE errors (exceeding 0.77) for models with $K = 10$ and $K = 50$, likely due to overfitting. Regularization did not improve performance on held-out data; for instance, increasing α slightly raised the validation MAE. We believe this is because the baseline model lacks the complexity required to capture the problem's underlying patterns.

The model with $K = 50$ achieved the lowest MAE on both validation and test sets among the three configurations. Although increasing K to 100 resulted in marginally better performance, **we opted for $K = 50$** as it provides a good balance between accuracy and computational efficiency.

Problem 2: Open-Ended Recommendation Challenge

2A:

To improve recommendation performance from the baseline methods in Problem 1, our proposed method is an **ensemble model combining SVD** (Singular Value Decomposition) and **KNNWithMeans** from the Surprise library to predict the ratings. The main idea behind choosing this method was to use the strengths of both models to compensate for their weaknesses and improve performance. This ensemble method aims to boost recommendation accuracy.

We loaded the training ratings from ratings_all_development_set.csv. We then performed hyperparameter tuning for both models separately using RandomizedSearchCV from the Surprise library to find the best parameters for each model. For the SVD model, we considered K values [20, 50,100, 150] and learning rate values [0.01, 0.02, 0.03, 0.04, 0.05, 0.1]. We considered the alpha values [0.05, 0.1, 0.15, 0.2, 0.5] for regularization. We tried a number of epochs [20, 30, 50, 100] epochs. These values were chosen based on prior experience with these models and the goal of balancing model complexity and performance. We searched with [3, 5, 10] cross-validation folds, ultimately selecting **5-fold validation**.

For the KNNWithMeans model, we tune the number of neighbors (k) and the similarity measure. We tested the k values [20, 30, 40, 50] along with the similarity measures like [cosine, Pearson, msd] correlation. Like SVD, we searched with [3, 5, 10] folds and chose 5.

After obtaining the best parameters for each model from the grid search, we used the best model for both SVD and KNN to generate predictions for the test set. Instead of relying on the predictions from each model independently, we combined them through **weighted averaging** to generate weighted predictions. We specifically used the weight of **0.6** for SVD predictions and **0.4** for KNN with means predictions. This **weighted averaging ensemble approach performed better** than an unweighted average because the SVD model outperformed the KNN model during cross-validation. The final predictions are then outputted for submission.

This reproducible method includes data loading, model training, hyperparameter tuning, and prediction generation. SVD captures latent factors by performing matrix factorization on the user-item rating matrix. It identifies underlying relationships between users and items, which helps predict ratings for unseen user-item pairs. Based on past interactions, SVD learns the hidden structure within the data, such as user preference or items' characteristics. KNNWithMeans, on the other hand, is a similarity-based algorithm that predicts ratings by finding similar users or items and averaging their ratings. It assumes that users who rated items similarly will rate them similarly in the future. Combining both methods in an ensemble approach, we optimize SVD's ability to capture complex latent structures and KNNWithMeans' strength in using similarity-based relationships. Overall, this ensemble approach helps to leverage the strength of both algorithms, with SVD capturing latent factors and KNNWithMeans focusing on similarity-based prediction.

2B:

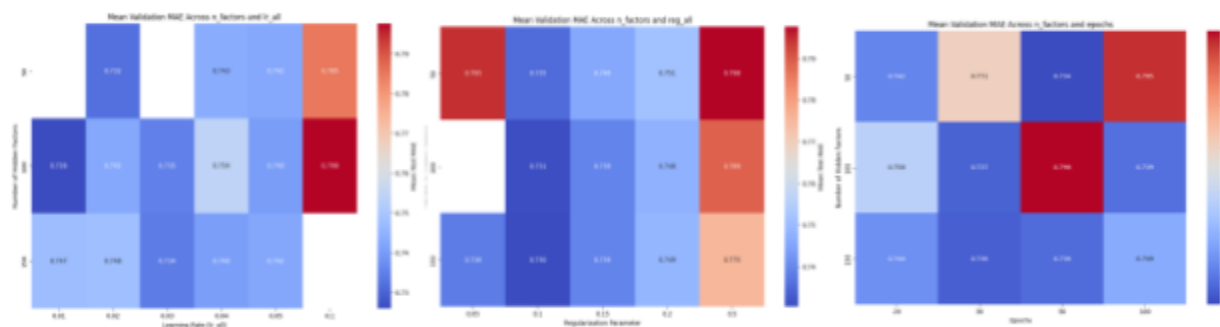


Figure 2a: The above heatmaps show the relationship between the hyperparameters chosen for model selection using RandomizedSearchCV for SVD. Each heatmap shows the interaction between the number of latent factors K with learning rate, regularization value, and number of epochs, respectively. The heatmaps suggest that $K = 100$ consistently yields better results. The figures also show that $lr = 0.01$, regularization term = 0.1, and 30-50 epochs are most likely the optimal parameters. We confirmed this by investigating the `cv_results` dataframe from the grid search.

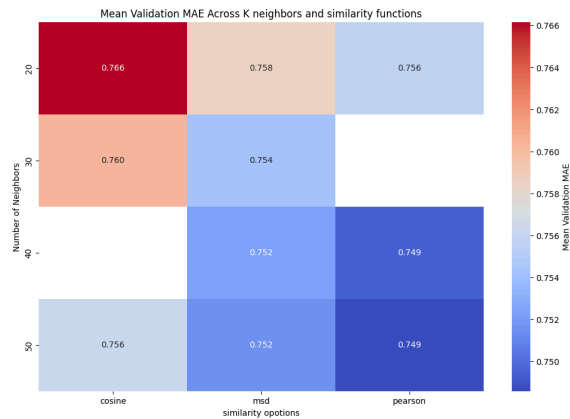


Figure 2b: The heatmap above shows the hyperparameter selection for KNNWithMeans using RandomizedSearchCV. As indicated by the map, the best validation error of 0.749 was achieved with K = 50 and the Pearson similarity function.

2C:

	Split 1	Split 2	Split 3	Split 4	Split 5	Mean MAE
SVD	0.729971	0.726633	0.715589	0.726355	0.731116	0.725933
KNN	0.746689	0.749339	0.747602	0.746875	0.752325	0.748576

Table 2: This table shows the validation MAE scores across 5 splits for the best models of SVD and KNN. Here are their respective leaderboard scores and final ensemble model's leaderboard MAE score:

- **SVD leaderboard score: 0.819**
- **KNN leaderboard score: 0.821**
- **(Final) Ensemble Model Leaderboard MAE score: 0.8165**

2D:

The table below shows the top ten **bad predictions** from our 5 fold cross-validation process. We investigate the users and movies associated with these bad predictions (uid is the user_id, iid is the item_id)

	fold	uid	iid	true_rating	predicted_rating	error
38260	2	886	257	1.0	5.000000	4.000000
37162	2	404	569	5.0	1.000000	4.000000
20901	1	404	573	5.0	1.037934	3.962066
57745	3	849	97	1.0	4.887072	3.887072
77147	4	294	182	1.0	4.886506	3.886506
73802	4	404	374	5.0	1.135185	3.864815
53622	2	313	172	1.0	4.820096	3.820096
51846	2	404	93	5.0	1.228604	3.771396
32616	1	238	63	1.0	4.762539	3.762539
41587	2	11	199	1.0	4.728963	3.728963

The first example involves **user 886** and **movie 257** (*Contact*, 1997). User 886, a 14-year-old female, previously rated *Aladdin* (1992) with 4 stars and *The Craft* (1996) with 5 stars. The model likely relied on simplistic patterns, such as associating the release years of *Aladdin* and *The Craft* with this user's preferences. However, this approach is flawed because *Contact* (1997) is a Sci-Fi movie, a genre distinct from the fantasy and animated genres of the other two films. This highlights a key limitation: the model's inability to incorporate richer contextual information, such as movie genres, themes, or other relevant attributes (e.g., cast, language, or popularity).

Interestingly, the same user rated *Back to the Future* (1985), another Sci-Fi movie, with 5 stars but gave *Contact* a 1-star rating. This discrepancy suggests that user preferences cannot be accurately inferred solely based on genres or release years, as individual tastes are influenced by more complex factors like the story, pacing, or personal connections to specific films. The model would need to learn and incorporate more nuanced features to improve.

User 404 is another interesting case, as it appears multiple times among the worst predictions. For example, they rated *Another Stakeout* (1993) and *City Slickers II: The Legend of Curly's Gold* (1994) with 5 stars, but the model failed to predict these ratings accurately. Examining their 1-star ratings reveals preferences such as *Land Before Time III: The Time of the Great Giving* (1995) and *The Magic Hour* (1998). Again, the model may have overly relied on superficial features like release years or titles, given that more relevant information, such as preferences for specific genres, directors, or casts, is missing from the data.

These examples demonstrate that successful prediction of user ratings requires deeper insights into both user preferences and movie attributes. User preferences are inherently complex and multi-faceted, influenced by factors beyond simple numerical data like release years or average ratings. Incorporating additional data such as movie metadata (genres, actors, language) and user-level behavioral patterns could significantly enhance the model's performance.

The table below shows 5 **good predictions** from our 5-fold cross-validation process. We investigate the users and movies associated with these good predictions.

	fold	uid	iid	true_rating	predicted_rating	error
3	0	393	163	4.0	3.972562	0.027438
5	0	560	481	4.0	3.934449	0.065551
19	0	55	41	4.0	4.095859	0.095859
22	0	495	37	2.0	2.090020	0.090020
31	0	59	1018	4.0	4.068983	0.068983

The model performs notably well for **user 393**, who has rated more than 50 movies. This suggests that users with more ratings provide the model with enough data to generalize their preferences effectively. Additionally, we observed that the model predicts better when the true ratings are in the 2-4 star range, compared to the extreme ratings (1 or 5) seen in previous analyses. This might indicate that mid-range ratings align more closely with general patterns in the dataset, whereas extreme ratings could reflect more individualized or subjective preferences that the model struggles to capture.

The model's MAE on held-out data is **0.8165**. The best SVD model's validation MAE is **0.7259**, where KNN's is **0.7486** across the 5 folds (see 2C). This is a noticeable gap between the validation and testing

errors, which suggests some degree of **overfitting to training data**; however, the underlying poor, overall performance (high MAE) hints at a broader limitation of the model's capacity, as we have previously mentioned its lack of more meaningful features that could help the model generalize better.

Compared to the model from **problem 1**, the two models exhibit similar validation MAE scores (around 0.72–0.73). It is worth mentioning that the test MAE reported for the model in problem 1 (~0.71) was based on the testing data from the `ratings_all_development_set.csv` file rather than the masked ratings used in problem 2. This likely indicates some overfitting to the training dataset in problem 1. Given that both models lack complexity, we expect the model from problem 1 would demonstrate a similar performance on truly held-out data as the model in problem 2. Both methods appear to struggle with capturing the nuances of user preferences and movie attributes, suggesting a need for more sophisticated modeling approaches.

2E: Other Attempted Models and Future Work:

During our model development, we explored several approaches but decided not to use them due to performance or implementation challenges. Initially, we tried using the **NMF** (non-negative matrix factorization) model from the Surprise library using the Surprise GridSearchCV module with parameters similar to the SVD model, but it resulted in slightly worse performance, with a mean validation MAE of 0.71 compared to SVD's 0.72. Thus, the model performed slightly worse than SVD on held-out data. Thus, we decided not to use this model.

We also tested various **KNN** models offered in Surprise, namely, KNNBasic, KNNBaseline, KNNwithMeans, and KNNZScore, again using GridSearchCV and optimizing parameters such as the number of neighbors k and the similarity function. The latter two models performed slightly better than the two former models. While KNNWithZScores gave us the best validation of 0.742, the performance still fell short compared to SVD. However, the MAE on validation data was around 0.73, which was still worse than that of SVD (0.72). We tried both GridSearchCV and RandomizedGridSearch for both models. The results were similar. However, a randomized grid search performed slightly better in SVD. We also tried **SVDPP**, but the performance was not improved since the model relies on implicit feedback (e.g., user interactions like clicks or views), which is absent in our dataset.

We also tried fitting a **linear regression model to make an ensemble from SVD and KNN** by making the predictions of SVD and KNN as our features for each user-item pair and the SVD predictions to be the y_{true} values, but the leaderboard error was slightly worse at 0.8199 compared to the weighted average of 0.8165.

Lastly, we were considering using LightFM, which is a library that uses hybrid models to train recommendation systems with both collaborative filtering and content filtering using user and movie info, but we did not have time to finish the implementation and encountered bugs trying to use the library. We predict that this model would have performed best since it would have considered user info like age and gender as well as movie info like title and release year, which would have particularly helped overcome the data sparsity problem and the complexity of this recommendation task.